

steps, and we shall go through each one of them. The *version* of Erlang/OTP can be passed as an argument to the playbook. Its default value is the release 19.0, and is defined in the variable section of the playbook as shown below:

```
vars:
  ERL_VERSION: "otp_src_{{ version | default('19.0') }}"
  ERL_DIR: "{{ ansible_env.HOME }}/installs/erlang"
  ERL_TOP: "{{ ERL_DIR }}/{{ ERL_VERSION }}"
  TEST_SERVER_DIR: "{{ ERL_TOP }}/release/tests/test_server"
```

The *ERL\_DIR* variable represents the directory where the tarball will be downloaded, and the *ERL\_TOP* variable refers to the top-level directory location containing the source code. The path to the test directory from where the tests will be invoked is given by the *TEST\_SERVER\_DIR* variable.

Erlang/OTP has mandatory and optional package dependencies. Let's first update the software package repository, and then install the required dependencies as indicated below:

```
tasks:
  - name: Update the software package repository
    become: true
    yum:
      name: '*'
      update_cache: yes

  - name: Install dependencies
    become: true
    package:
      name: "{{ item }}"
      state: latest
    with_items:
      - wget
      - make
      - gcc
      - perl
      - m4
      - ncurses-devel
      - sed
      - libxslt
      - fop
```

The Erlang/OTP sources are written using the 'C' programming language. The GNU C Compiler (GCC) and GNU Make are used to compile the source code. The 'libxslt' and 'fop' packages are required to generate the documentation. The build directory is then created, the source tarball is downloaded and it is extracted to the directory mentioned in *ERL\_DIR*.

```
- name: Create destination directory
  file: path="{{ ERL_DIR }}" state=directory
```

```
- name: Download and extract Erlang source tarball
  unarchive:
    src: "http://erlang.org/download/{{ ERL_VERSION }}.tar.
    gz"
    dest: "{{ ERL_DIR }}"
    remote_src: yes
```

The 'configure' script is available in the sources, and it is used to generate the Makefile based on the installed software. The 'make' command will build the binaries from the source code.

```
- name: Build the project
  command: "{{ item }}" chdir="{{ ERL_TOP }}"
  with_items:
    - ./configure
    - make
  environment:
    ERL_TOP: "{{ ERL_TOP }}"
```

After the 'make' command finishes, the 'bin' folder in the top-level sources directory will contain the Erlang 'erl' interpreter. The Makefile also has targets to run tests to verify the built binaries. We are remotely invoking the test execution from Ansible and hence *-noshell -noinput* are passed as arguments to the Erlang interpreter, as shown in the *.yaml* file.

```
- name: Prepare tests
  command: "{{ item }}" chdir="{{ ERL_TOP }}"
  with_items:
    - make release_tests
  environment:
    ERL_TOP: "{{ ERL_TOP }}"

- name: Execute tests
  shell: "cd {{ TEST_SERVER_DIR }} && {{ ERL_TOP }}/bin/erl
  -noshell -noinput -s ts install -s ts smoke_test batch -s
  init stop"
```

You need to verify that the tests have passed successfully by checking the *\$ERL\_TOP/release/tests/test\_server/index.html* page in a browser. A screenshot of the test results is shown in Figure 1.

The built executables and libraries can then be installed on the system using the *make install* command. By default, the install directory is */usr/local*.

```
- name: Install
  command: "{{ item }}" chdir="{{ ERL_TOP }}"
  with_items:
    - make install
  become: true
  environment:
    ERL_TOP: "{{ ERL_TOP }}"
```